

AD23B31 Image Processing and Computer Vision

**Image Processing Web Application using Flask
and OpenCV**

A PROJECT REPORT

Submitted by

2116230701108 – HARSHA VARDHANAN V



May2026

BONAFIDE CERTIFICATE

Certified that this project report titled “**Image Processing Web Application using Flask and OpenCV**” is the bonafide work carried out by **HARSHA VARDHANAN V (230701108)** under my supervision and guidance in partial fulfillment of the requirements for the Mini Project in the Department of Computer Science and Engineering during the academic year 2025 – 2026.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Report

2. LITERATURE REVIEW

- 2.1 Traditional Image Processing Methods
- 2.2 OpenCV-Based Systems
- 2.3 Existing Web-Based Image Editors
- 2.4 Gap Identified

3. SYSTEM DESCRIPTION

- 3.1 Project Overview
- 3.2 Features of the System
- 3.3 Technology Stack
- 3.4 System Architecture

4. PREPROCESSING AND IMAGE PROCESSING METHODOLOGY

- 4.1 Image Upload and Validation
- 4.2 Grayscale Conversion
- 4.3 Edge Detection
- 4.4 Gaussian Blur
- 4.5 Thresholding
- 4.6 Sharpening
- 4.7 Sepia Filter
- 4.8 Brightness and Contrast Adjustment

5. IMPLEMENTATION DETAILS

- 5.1 Frontend Design
- 5.2 Backend Design
- 5.3 Flask API Routes
- 5.4 Image Storage and Retrieval
- 5.5 Error Handling

6.RESULTS AND ANALYSIS

6.1 Functional Results

6.2 Filter Performance Analysis

6.3 User Interface Evaluation

7.DISCUSSION

8.CONCLUSION AND FUTURE WORK

9.REFERENCES

10.APPENDIX A — SOURCE CODE

11.APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

ABSTRACT

Image processing plays an important role in modern computer vision applications by enabling computers to analyze, enhance, and manipulate digital images efficiently. This project presents a full-stack Image Processing Web Application that allows users to perform various image processing operations through an interactive and user-friendly web interface. The system is developed using Flask for the backend and HTML, CSS, and JavaScript for the frontend, while OpenCV is used as the core image processing library.

The application enables users to upload images and apply multiple processing techniques such as grayscale conversion, edge detection, Gaussian blur, thresholding, sharpening, horizontal flipping, sepia filtering, and brightness and contrast adjustment. The system processes images on the server side and displays the processed output instantly within the browser. Additional features such as before-and-after comparison, image reset functionality, and processed image download support improve user interaction and usability.

The proposed system demonstrates the practical integration of web technologies and computer vision algorithms in a lightweight and beginner-friendly environment. The application reduces the complexity of traditional image editing software by providing a simple platform for experimenting with image enhancement techniques. The project also highlights the effectiveness of OpenCV for implementing real-time image manipulation operations in web applications.

Overall, the project serves as an educational and practical demonstration of image processing concepts and web-based system development. Future improvements may include AI-based image enhancement, webcam support, cloud deployment, and advanced deep learning filters for intelligent image editing applications.

1. INTRODUCTION

1.1 Background and Motivation

Image processing is an important area of computer vision that focuses on manipulating and analyzing digital images using computational techniques. With the rapid growth of multimedia applications, image editing and enhancement tools have become essential in fields such as photography, healthcare, surveillance, education, and entertainment.

Traditional desktop-based image editing software often requires installation, high system resources, and technical knowledge. Web-based image processing applications provide a lightweight and accessible alternative, enabling users to upload, process, and download images directly from a browser without installing complex software.

This project is motivated by the need to develop a simple and beginner-friendly image processing application that demonstrates the practical implementation of computer vision techniques using OpenCV and Flask.

1.2 Problem Statement

Many existing image editing applications are either too complex for beginners or require high-end software installations. There is a need for a lightweight web application that allows users to perform basic image processing operations such as filtering, enhancement, and edge detection in a simple and interactive environment.

1.3 Objectives

The objectives of this project are:

- To develop a web-based image processing application using Flask and OpenCV.
- To implement commonly used image processing filters and enhancement techniques.
- To provide real-time image preview and processing functionality.
- To enable users to upload, process, compare, and download images easily.
- To design a responsive and beginner-friendly user interface.

1.4 Scope of the Report

This report focuses on the design and implementation of a web-based image processing system using Python, Flask, and OpenCV. The system supports various image filters such as grayscale, edge detection, blur, thresholding, sharpening, and sepia effects. The report includes system architecture, implementation details, image processing techniques, and performance evaluation. Advanced AI-based image editing and cloud deployment are beyond the scope of this project.

2. LITERATURE REVIEW

Image processing and computer vision systems have become widely used in modern software applications. Researchers and developers have proposed various approaches for implementing image manipulation and enhancement systems.

2.1 Traditional Image Processing Methods

Traditional image processing methods involve mathematical operations on pixel values to enhance image quality or extract meaningful information. Common techniques include grayscale conversion, filtering, thresholding, and edge detection.

These methods are computationally efficient and are still widely used in educational and industrial applications.

2.2 OpenCV-Based Systems

OpenCV is one of the most popular open-source libraries used for computer vision and image processing tasks. It provides optimized implementations for image filtering, feature extraction, object detection, and image transformations.

Many image editing systems use OpenCV because of its speed, simplicity, and cross-platform compatibility.

2.3 Existing Web-Based Image Editors

Several web-based image editors such as Canva, Pixlr, and Fotor provide online image editing functionalities. These platforms support advanced editing tools and cloud-based services.

However, many existing applications are either feature-heavy or require user accounts and subscriptions. Educational image processing projects aim to provide simpler alternatives focused on learning and experimentation.

2.4 Gap Identified

Although many image editing applications are available, beginner-friendly systems demonstrating the integration of Flask and OpenCV for educational purposes are limited. Many platforms also lack transparency in how image processing algorithms work internally. This project addresses these gaps by developing a simple and understandable image processing web application with clearly implemented filters and real-time interaction.

3. SYSTEM DESCRIPTION

3.1 Project Overview

The Image Processing Web App is a browser-based application that allows users to upload images and apply different image processing operations using OpenCV. The application processes images on the server side using Flask and displays the results instantly on the webpage.

The system supports multiple image filters, brightness and contrast adjustment, before-and-after comparison, and image download functionality.

3.2 Features of the System

The main features of the system include:

- Image Upload
- Real-time Image Preview
- Grayscale Conversion
- Edge Detection
- Gaussian Blur
- Horizontal Flip
- Binary Thresholding
- Sharpening Filter
- Sepia Effect
- Brightness and Contrast Adjustment
- Reset Functionality
- Download Processed Image
- Before/After Comparison

3.3 Technology Stack

The following technologies are used in this project:

Component	Technology Used
Frontend	HTML5, CSS3, JavaScript
Backend	Python Flask
Image Processing	OpenCV
Numerical Computation	NumPy
Communication	Fetch API

3.4 System Architecture

The system architecture consists of three major components:

1.Frontend Interface

The frontend provides a responsive user interface where users can upload and process images.

2.Flask Backend

The backend handles HTTP requests, image uploads, filter application, and processed image generation.

3.OpenCV Processing Engine

OpenCV performs image manipulation operations and returns processed outputs.

The workflow of the system is as follows:

1. User uploads image
2. Flask receives image request
3. OpenCV processes image
4. Processed image is stored
5. Result is displayed to user
6. User downloads processed image

4. MODULES DESCRIPTION

4.1 Image Upload and Validation

The application accepts JPG, JPEG, and PNG image formats. Uploaded images are validated before processing to ensure compatibility and prevent errors.

Invalid file formats are rejected with an error message.

4.2 Grayscale Conversion

Grayscale conversion transforms a colored image into shades of gray by reducing RGB channels into intensity values.

This operation simplifies image analysis and is commonly used in computer vision applications.

4.3 Edge Detection

Edge detection identifies boundaries and structural features within an image. The Canny edge detection algorithm is used in this project.

The process includes:

- Noise reduction
- Gradient calculation
- Edge tracking

Edge detection helps highlight object outlines and image structures.

4.4 Gaussian Blur

Gaussian blur is used to smooth images and reduce noise. The filter applies a Gaussian kernel to average neighboring pixel values.

This technique is commonly used before edge detection and object recognition tasks.

4.5 Thresholding

Thresholding converts grayscale images into binary images by assigning pixels either black or white based on intensity values.

This operation is useful for segmentation and feature extraction.

4.6 Sharpening

Sharpening enhances image details by emphasizing edges and textures. A sharpening kernel is applied using convolution operations.

The filter improves image clarity and visibility.

4.7 Sepia Filter

The sepia filter applies a warm brown tone to images, creating a vintage effect.

The transformation modifies RGB values using weighted combinations.

4.8 Brightness and Contrast Adjustment

Brightness and contrast controls allow users to modify image appearance interactively.

Brightness changes pixel intensity values, while contrast adjusts the difference between dark and light regions.

The application provides slider controls for real-time adjustments.

5. IMPLEMENTATION AND RESULTS

5.1 Frontend Design

The frontend is developed using HTML, CSS, and JavaScript.

Key frontend components include:

- Image upload section
- Filter buttons
- Image preview panels
- Adjustment sliders
- Download button
- Reset functionality

AJAX-based communication using the Fetch API enables smooth interactions without page reloads.

5.2 Backend Design

The backend is implemented using Flask, a lightweight Python web framework.

The backend performs the following tasks:

- Handles image uploads
- Applies OpenCV filters
- Stores processed outputs
- Returns image paths to frontend
- Manages reset and download functionality

5.3 Flask API Routes

The application contains the following API routes:

Route Function

GET / Load homepage

POST /upload Upload image

POST /process/filter Apply image filter

POST /adjust Adjust brightness and contrast

POST /reset Reset image

GET /uploads/file Retrieve uploaded image

GET /outputs/file Retrieve processed image

5.4 Image Storage and Retrieval

Uploaded images are stored inside the uploads directory, while processed images are stored inside the outputs directory.

The application dynamically retrieves images using Flask routes.

5.5 Error Handling

The system includes error handling mechanisms for:

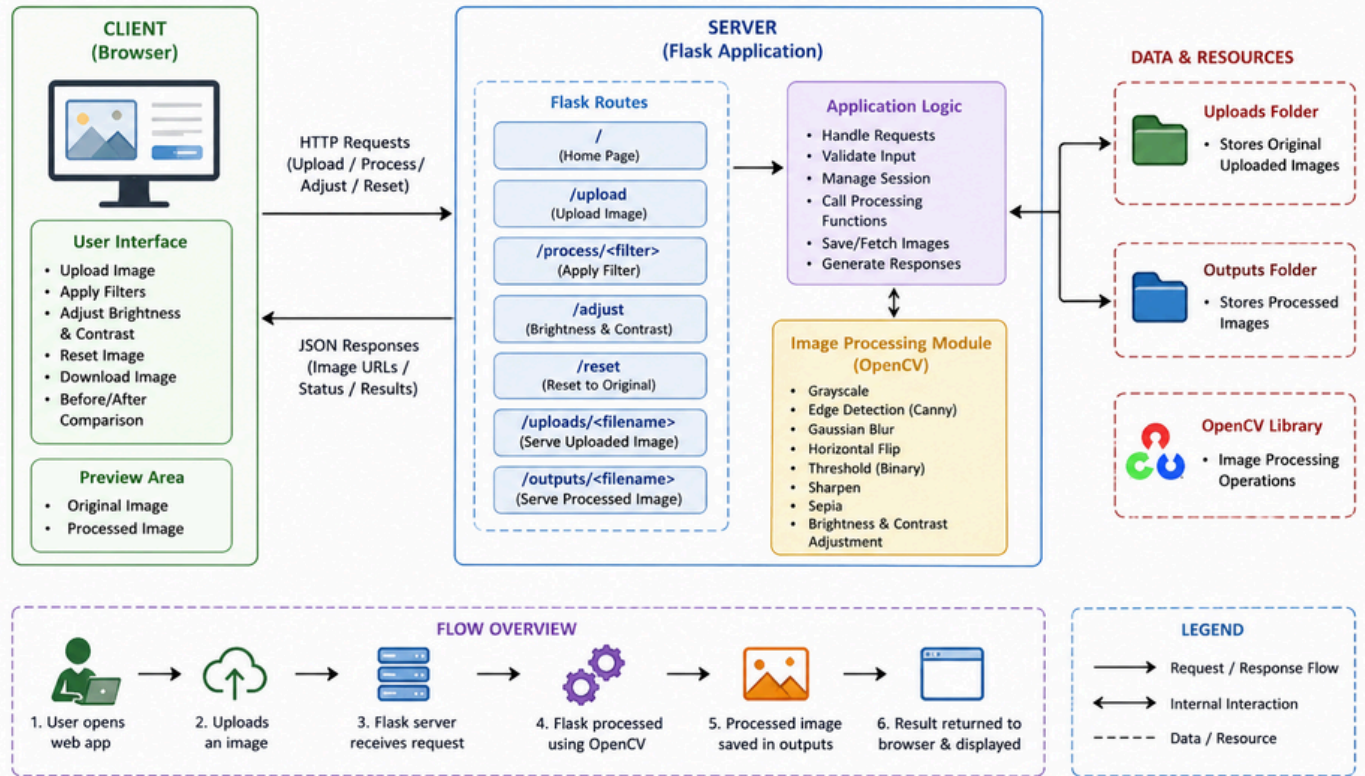
- Invalid image formats
- Missing image uploads
- File read failures
- Empty processing requests

These mechanisms improve system reliability and user experience.

Table 3: Implementation Configuration

Parameter	Value
Project	Image Processing Web App
Framework	Flask Web Framework
Backend Language	Python 3.10
Frontend Technologies	HTML5, CSS3, JavaScript
Image Processing Library	OpenCV (cv2)
Numerical Computation	NumPy
Supported Image Formats	JPG, JPEG, PNG
Implemented Filters	Grayscale, Edge Detection, Blur, Flip, Threshold, Sharpen, Sepia
Adjustment Features	Brightness and Contrast Control
Communication Method	Fetch API (AJAX Requests)
Image Storage	Uploads and Outputs Folders
Web Architecture	Client-Server Architecture
Processing Type	Server-side Image Processing
Development Environment	Visual Studio Code
Operating System	Windows 10 / 11
Browser Support	Google Chrome, Microsoft Edge
Additional Features	Real-time Preview, Download, Reset, Before/After Comparison
Hardware Requirements	Min. 4GB RAM, Dual-Core Processor
Libraries / Frameworks	Flask, OpenCV, NumPy, Werkzeug

ARCHITECTURE DIAGRAM – IMAGE PROCESSING WEB APP



6. RESULTS AND ANALYSIS

6.1 Functional Results

The developed application successfully performs all implemented image processing operations.

Users can:

- Upload images successfully
- Apply filters instantly
- Compare original and processed outputs
- Adjust brightness and contrast
- Download processed images

The system demonstrates smooth interaction between the frontend and backend.

6.2 Filter Performance Analysis

Table 1 presents the implemented filters and their functionality.

Table 1: Image Processing Filters

Filter Purpose

Grayscale Convert image to black and white

Edge Detection Highlight image edges

Gaussian Blur Reduce image noise

Thresholding Convert image to binary

Sharpen Enhance image details

Sepia Apply vintage effect

Flip Mirror image horizontally

The filters produced visually accurate results for different image types.

6.3 User Interface Evaluation

The user interface is designed to be simple, responsive, and beginner-friendly.

The before-and-after comparison feature improves usability by allowing users to analyze processing effects easily.

The application also works effectively on different screen sizes.

7. DISCUSSION

The Image Processing Web App demonstrates the successful integration of Flask and OpenCV for implementing real-time image processing operations in a web environment.

One of the major strengths of the system is its simplicity. Users can perform image enhancement operations without installing specialized software. The lightweight design also makes the application suitable for educational demonstrations and beginner projects.

The use of OpenCV provides efficient implementations for image filtering and manipulation. The Flask framework simplifies backend development and API management.

The application successfully processes images with low latency and provides immediate visual feedback. Features such as real-time preview, download support, and brightness adjustment improve the overall user experience.

However, the project has certain limitations. Large image files may increase processing time due to server-side computation. The system also supports only basic image processing operations and does not include advanced AI-based enhancements.

Despite these limitations, the project effectively demonstrates practical applications of computer vision and web development technologies.

8. CONCLUSION AND FUTURE WORK

This mini project report presented the design and implementation of a web-based image processing application using Flask and OpenCV.

The application successfully performs multiple image processing operations including grayscale conversion, edge detection, blurring, thresholding, sharpening, sepia filtering, and brightness adjustment.

The project demonstrates how computer vision techniques can be integrated with web technologies to create interactive and user-friendly applications.

The key contributions of this project are:

- Development of a complete web-based image processing application using Flask and OpenCV.
- Implementation of multiple image filters and enhancement techniques.
- Integration of frontend and backend communication using Fetch API.
- Responsive and beginner-friendly user interface design.
- Real-time image processing and preview functionality.

Future work directions include:

- Adding AI-based image enhancement techniques.
- Supporting real-time webcam image processing.
- Integrating cloud storage for uploaded images.
- Developing mobile-responsive progressive web application support.
- Implementing advanced filters using deep learning models.

REFERENCES

- [1] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.
- [2] A. Kaehler and G. Bradski, Learning OpenCV: Computer Vision with the OpenCV Library, O’Reilly Media, 2008.
- [3] M. Nixon and A. Aguado, Feature Extraction and Image Processing for Computer Vision, Academic Press, 2019.
- [4] Flask Documentation, “Flask Web Framework,” [Online]. Available: <https://flask.palletsprojects.com/>
- [5] NumPy Documentation, “NumPy Library,” [Online]. Available: <https://numpy.org/>
- [6] R. Szeliski, Computer Vision: Algorithms and Applications, Springer, 2011.
- [7] R. Gonzalez and R. Woods, Digital Image Processing, Pearson Education, 2018.
- [8] OpenCV Documentation, “Image Processing Techniques,” [Online]. Available: <https://opencv.org/>
- [9] Mozilla Developer Network, “Fetch API,” [Online]. Available: <https://developer.mozilla.org/>

APPENDIX A — SOURCE CODE

```
# =====  
# Image Processing Web App using Flask  
# and OpenCV  
# =====  
  
# Import Required Libraries  
from flask import Flask, request, jsonify  
import cv2  
import numpy as np  
import os  
  
# Flask App Configuration  
app = Flask(__name__)
```



```

UPLOAD_FOLDER = "uploads"
OUTPUT_FOLDER = "outputs"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
app.config["OUTPUT_FOLDER"] = OUTPUT_FOLDER
# =====
# IMAGE UPLOAD ROUTE
# =====
@app.route("/upload", methods=["POST"])
def upload_image():
    image_file = request.files["image"]
    # Save uploaded image
    image_path = os.path.join(
        app.config["UPLOAD_FOLDER"],
        image_file.filename
    )
    image_file.save(image_path)
    return jsonify({
        "message": "Image uploaded successfully"
    })
# =====
# FILTER FUNCTION
# =====
def apply_filter(image, filter_name):
    if filter_name == "grayscale":
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    elif filter_name == "edge":
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return cv2.Canny(gray, 100, 200)
    elif filter_name == "blur":
        return cv2.GaussianBlur(image, (15, 15), 0)
    elif filter_name == "flip":
        return cv2.flip(image, 1)
    elif filter_name == "threshold":
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        _, binary = cv2.threshold(
            gray, 128, 255, cv2.THRESH_BINARY
        )
    return binary

```

```

elif filter_name == "sharpen":
    kernel = np.array([
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0]
    ])
    return cv2.filter2D(image, -1, kernel)
    return image
# =====
# BRIGHTNESS AND CONTRAST
# =====
def adjust_brightness_contrast(
    image,
    brightness=0,
    contrast=1.0
):
    adjusted = cv2.convertScaleAbs(
        image,
        alpha=contrast,
        beta=brightness
    )
    return adjusted
# =====
# PROCESS IMAGE ROUTE
# =====
@app.route("/process/<filter_name>", methods=["POST"])
def process_image(filter_name):
    image = cv2.imread("sample.jpg")
    processed = apply_filter(image, filter_name)
    output_path = os.path.join(
        app.config["OUTPUT_FOLDER"],
        "processed.jpg"
    )
    cv2.imwrite(output_path, processed)
    return jsonify({
        "message": "Filter Applied Successfully"
    })
# =====
# MAIN FUNCTION
# =====
if __name__ == "__main__":
    app.run(debug=True)

```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

Basic Image Filter App

Choose an image (jpg, jpeg, png) 1777811711990_198f8f...39b701dbab38b117.jpg

sepia applied successfully. You can download the result.

Brightness: 0 Contrast: 1.0

Original Image



Processed Image



Basic Image Filter App

Choose an image (jpg, jpeg, png)

Choose File

1777811711990_1988f...39b701dub38b117.jpg

Upload Image

edge applied successfully. You can download the result.

Brightness: -24

Contrast: 1.5

Apply

Original Image



Processed Image



Grayscale

Edge Detection

Gaussian Blur

Horizontal Flip

Threshold

Sharpen

Sepia

Reset Image

Show Original

Download Processed Image

Basic Image Filter App

Choose an image (jpg, jpeg, png)

Choose File

1777811711990_1988f...39b701dub38b117.jpg

Upload Image

grayscale applied successfully. You can download the result.

Brightness: -24

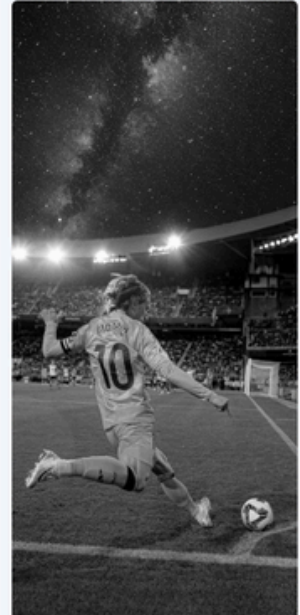
Contrast: 1.5

Apply

Original Image



Processed Image



Grayscale

Edge Detection

Gaussian Blur

Horizontal Flip

Threshold

Sharpen

Sepia

Reset Image

Show Original

Download Processed Image

Choose an image (jpg, jpeg, png)

Choose File

1777811711990_1988f...39b701dub38b117.jpg

Upload Image

flip applied successfully. You can download the result.

Brightness: -24

Contrast: 1.5

Apply

Original Image



Processed Image



Grayscale

Edge Detection

Gaussian Blur

Horizontal Flip

Threshold

Sharpen

Sepia

Reset Image

Show Original

Download Processed Image